10.2 Ollama API

For automated code processing, you can also use a local LLM instead of a commercial language model. This requires a reasonably powerful computer. Depending on which software you use to execute the language model, there are various modules and APIs for executing prompts in Python scripts. Here we're focusing on the Ollama Python library. The module requires Ollama to be running on the local computer or on a server accessible on the network.

It's best to set up a Python environment directory for initial experiments. The following commands apply to Linux and macOS:

```
python3 -m venv ollama-test
cd ollama-tests
source bin/activate
pip3 install ollama
```

The following Hello World example assumes that Ollama is running on your computer and that the `llama3` language model is installed. Using the Ollama API is very similar to using the OpenAI API, at least as far as the basic functions are concerned. Of course, this is no coincidence. Various modules follow the OpenAI concept to make switching between different language models and libraries as easy as possible.

```
# Sample file ollama/hello.py
import ollama
response = ollama.chat(
  model="llama3",
  options = {
    "temperature": 0.5,
    "num_predict": 200 },  # corresponds to max_tokens
with OpenAI
  messages = [
```

```
    { "role": "system",
      "content": "You are a programming assistant."},
    { "role": "user",
      "content": "How do I use list comprehension in
Python?"} ])

print("Done:          ", response["done"])  #
True/False
# "done_reason" in Ollama corresponds to "stop" in
OpenAI
print("Done reason:  ", response["done_reason"])
print("Output tokens:", response["eval_count"])
print("Response message:")
print(response["message"]["content"])
```

Further details on the `ollama` module and the underlying API can be found here:

- **https://github.com/ollama/ollama**
- **https://github.com/ollama/ollama/blob/main/docs/api.md**

10.2.1 Manipulating Code

Like OpenAI, Ollama doesn't provide for the direct editing of files. And as with OpenAI, you can also achieve this goal with Ollama by means of a small workaround: for this purpose, you need to embed the content of the file in a prompt and save the result in a new file. As expected, the code looks very similar to the corresponding OpenAI example:

```
# Sample file ollama/hello-upload.py
import ollama

# Read file to be edited
with open('sample.py', 'r') as file:
    file_contents = file.read()

# Put together roll and prompt
role = """You are a programming assistant. You analyze
code
  and return an improved version of it. Organize code
into
```

```
    functions where possible. Include code comments
where
    necessary.
    You return code ONLY!
    DO NOT ADD EXPLANATIONS.
    DO NOT USE MARKDOWN to format your output."""
prompt = f"Optimize the following
code:\n\n{file_contents}"

# Execute prompt
response = ollama.chat(
    model="gemma2",
    options = { "temperature": 0.5 },
    messages = [ {"role": "system", "content": role},
                 {"role": "user",   "content": prompt}
] )

if response["done_reason"] != "stop":
    print("Code optimization failed.")
else:
    new_code = response["message"]["content"].strip()
    with open('sample-improved.py', 'w') as file:
        file.write(new_code)
    print("\nImproved code\n\n")
    print(new_code)
```

The formatting of the result proved to be the biggest issue. Many free LLMs weren't able (as of the time of writing) to follow the instructions clearly formulated in the role description. We've tested the code with various LLMs:

- `gemma2` works perfectly.
- `llama3` and `codestral`: Both models often resist the desire to dispense with Markdown. The code begins and ends with ```. But after all, these models don't add any explanations. It would be relatively easy to eliminate the ```` lines using a post-processing function.
- `codellama`: The answer is often made up of the code plus various explanations. Before further automated processing can start, the code and the explanations must be separated from each other, which is difficult.

- `starcoder2:3b` often doesn't understand the task and gives inappropriate answers.